

Collections Framework

Collections in java is a framework that provides an architecture to store and manipulate the group of objects. All the operations that you perform on a data such as searching, sorting, insertion, manipulation, deletion etc. can be performed by Java Collections. Java Collection simply means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque etc.) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet etc).

What is Collection in java

Collection represents a single unit of objects i.e. a group.

What is framework in java

A framework is a set of classes and interfaces which provide a ready-made architecture. In order to implement a new feature or a class, there is no need to define a framework. However, an optimal object-oriented design always includes a framework with a collection of classes such that all the classes perform the same kind of task.

- o provides readymade architecture.
- o represents set of classes and interface.
- o is optional.

What is Collection framework

Any group of individual objects which are represented as a single unit is known as a collection of objects. In Java, a separate framework named the “*Collection Framework*” has been defined in JDK which holds all the collection classes and interface in it. In Java, Collection interface (**java.util.Collection**) and Map interface (**java.util.Map**) are the two main “root” interfaces of Java collection classes. Collection framework represents a unified architecture for storing and manipulating group of objects. It has:

1. Interfaces and its implementations i.e. classes
2. Algorithm

Advantages of the Collection Framework

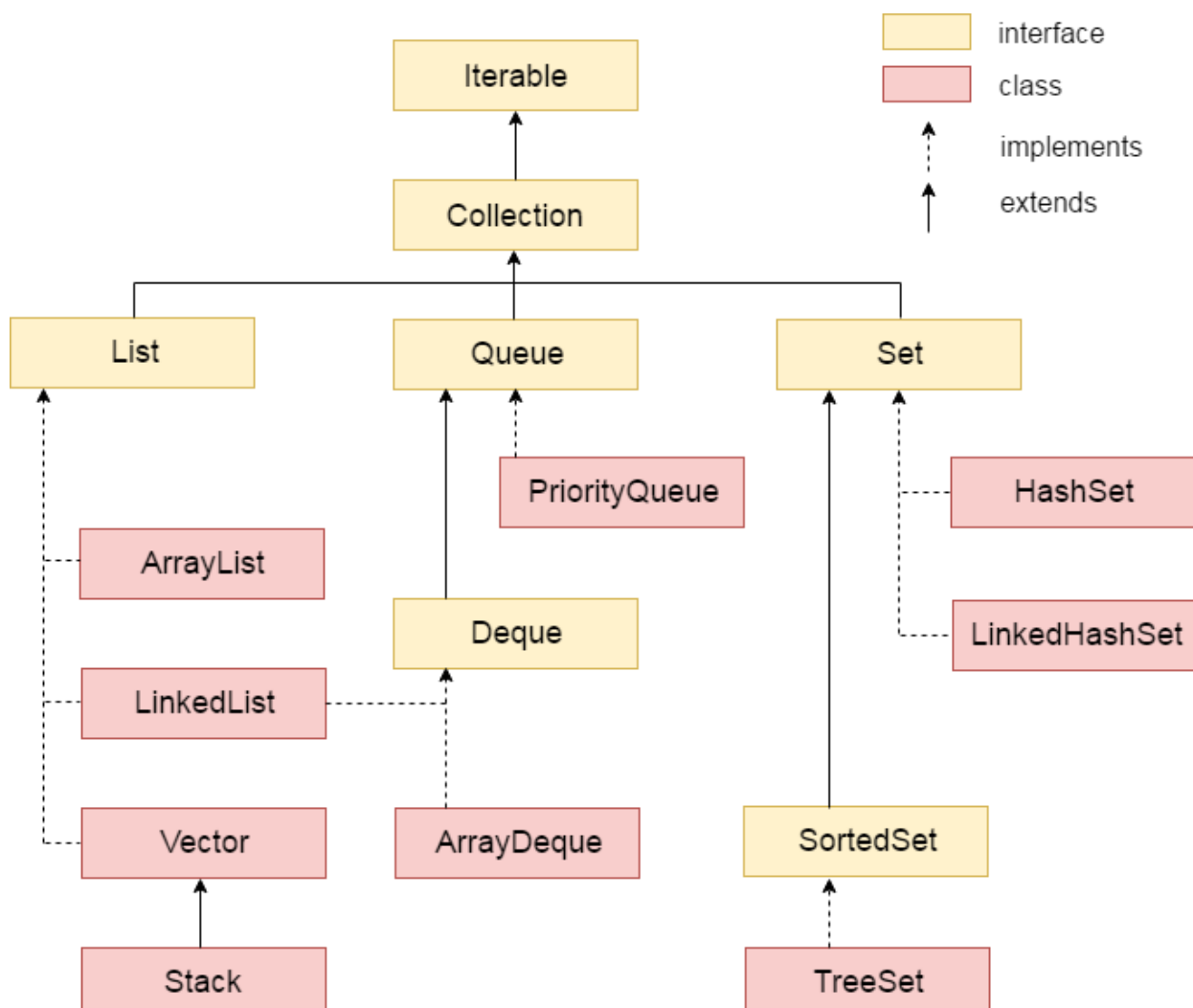
the following are the advantages of the collection framework.

1. **Consistent API:** The API has a basic set of interfaces like *Collection*, *Set*, *List*, or *Map*, all the classes (ArrayList, LinkedList, Vector, etc) that implement these interfaces have *some* common set of methods.

2. **Reduces programming effort:** A programmer doesn't have to worry about the design of the Collection but rather he can focus on its best use in his program. Therefore, the basic concept of Object-oriented programming (i.e.) abstraction has been successfully implemented.
3. **Increases program speed and quality:** Increases performance by providing high-performance implementations of useful data structures and algorithms because in this case, the programmer need not think of the best implementation of a specific data structure.

Hierarchy of the Collection Framework

The utility package, (java.util) contains all the classes and interfaces that are required by the collection framework. The collection framework contains an interface named an iterable interface which provides the iterator to iterate through all the collections. This interface is extended by the main collection interface which acts as a root for the collection framework. All the collections extend this collection interface thereby extending the properties of the iterator and the methods of this interface. The following figure illustrates the hierarchy of the collection framework.



Before understanding the different components in the above framework, let's first understand a class and an interface.

- **Class**: A class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type.
- **Interface**: Like a class, an interface can have methods and variables, but the methods declared in an interface are by default abstract (only method signature, nobody). Interfaces specify what a class must do and not how. It is the blueprint of the class.

Interfaces that extend the Collections Interface

1. Iterable Interface

This is the root interface for the entire collection framework. The collection interface extends the iterable interface. Therefore, inherently, all the interfaces and classes implement this interface. The main functionality of this interface is to provide an iterator for the collections. Therefore, this interface contains only one abstract method which is the iterator.

Iterator iterator();

It returns the iterator over the elements of type T.

2. Iterator interface

Iterator interface provides the facility of iterating the elements in a forward direction only.

Methods of Iterator interface

There are only three methods in the Iterator interface. They are:

No.	Method	Description
1	public boolean hasNext()	It returns true if the iterator has more elements otherwise it returns false.
2	public Object next()	It returns the element and moves the cursor pointer to the next element.
3	public void remove()	It removes the last elements returned by the iterator. It is less used.

3. Collection Interface

This interface extends the iterable interface and is implemented by all the classes in the collection framework. This interface contains all the basic methods which every collection has like adding the data into the collection, removing the data, clearing the data, etc. All these methods are implemented in this interface because these methods are implemented by all the classes irrespective of their style of implementation. And also, having these methods in this interface ensures that the names of the methods are universal for all the collections. Therefore, in short, we can say that this interface builds a foundation on which the collection classes are implemented.

Methods of Collection interface

There are many methods declared in the Collection interface. They are as follows:

No.	Method	Description
1	public boolean add(E e)	It is used to insert an element in this collection.
2	public boolean addAll(Collection<? extends E> c)	It is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object element)	It is used to delete an element from the collection.
4	public boolean removeAll(Collection<?> c)	It is used to delete all the elements of the specified collection from the invoking collection.
5	default boolean removeIf(Predicate<? super E> filter)	It is used to delete all the elements of the collection that satisfy the specified predicate.
6	public boolean retainAll(Collection<?> c)	It is used to delete all the elements of invoking collection except the specified collection.
7	public int size()	It returns the total number of elements in the collection.
8	public void clear()	It removes the total number of elements from the collection.
9	public boolean contains(Object element)	It is used to search an element.
10	public boolean containsAll(Collection<?> c)	It is used to search the specified collection in the collection.
11	public Iterator iterator()	It returns an iterator.
12	public Object[] toArray()	It converts collection into array.
13	public <T> T[] toArray(T[] a)	It converts collection into array. Here, the runtime type of the returned array is that of the specified array.

14	public boolean isEmpty()	It checks if collection is empty.
15	default Stream<E> parallelStream()	It returns a possibly parallel Stream with the collection as its source.
16	default Stream<E> stream()	It returns a sequential Stream with the collection as its source.
17	default Spliterator<E> spliterator()	It generates a Spliterator over the specified elements in the collection.
18	public boolean equals(Object element)	It matches two collections.
19	public int hashCode()	It returns the hash code number of the collection.

4. List Interface

This is a child interface of the collection interface. This interface is dedicated to the data of the list type in which we can store all the ordered collections of the objects. This also allows duplicate data to be present in it. This list interface is implemented by various classes like ArrayList, Vector, Stack, etc. Since all the subclasses implement the list, we can instantiate a list object with any of these classes.

For example:

```
List <T> al = new ArrayList<> ();
```

```
List <T> ll = new LinkedList<> ();
```

```
List <T> v = new Vector<> ();
```

Where T is the type of the object

5. Queue Interface

As the name suggests, a queue interface maintains the FIFO(First In First Out) order similar to a real-world queue line. This interface is dedicated to storing all the elements where the order of the elements matter. For example, whenever we try to book a ticket, the tickets are sold on a first come first serve basis. Therefore, the person whose request arrives first into the queue gets the ticket. There are various classes like [PriorityQueue](#), [ArrayDeque](#), etc. Since all these subclasses implement the queue, we can instantiate a queue object with any of these classes.

For example:

```
Queue <T> pq = new PriorityQueue<> ();
```

```
Queue <T> ad = new ArrayDeque<> ();
```

Where T is the type of the object.

6. Deque Interface

This is a very slight variation of the [queue data structure](#). Deque, also known as a double-ended queue, is a data structure where we can add and remove elements from both ends of the queue. This interface extends the queue interface. The class which implements this interface is ArrayDeque. Since ArrayDeque class implements the Deque interface, we can instantiate a deque object with this class.

For example:

```
Deque<T> ad = new ArrayDeque<> ();
```

Where T is the type of the object.

7. Set Interface

A set is an unordered collection of objects in which duplicate values cannot be stored. This collection is used when we wish to avoid the duplication of the objects and wish to store only the unique objects. This set interface is implemented by various classes like HashSet, TreeSet, LinkedHashSet, etc. Since all the subclasses implement the set, we can instantiate a set object with any of these classes.

For example:

```
Set<T> hs = new HashSet<> ();
```

```
Set<T> lhs = new LinkedHashSet<> ();
```

```
Set<T> ts = new TreeSet<> ();
```

Where T is the type of the object.

8. Sorted Set Interface

This interface is very similar to the set interface. The only difference is that this interface has extra methods that maintain the ordering of the elements. The sorted set interface extends the set interface and is used to handle the data which needs to be sorted. The class which implements this interface is TreeSet. Since this class implements the SortedSet, we can instantiate a SortedSet object with this class.

For example:

```
SortedSet<T> ts = new TreeSet<> ();
```

Where T is the type of the object.

9. Map Interface

A map is a data structure that supports the key-value pair for mapping the data. This interface doesn't support duplicate keys because the same key cannot have multiple mappings, however, it allows duplicate values in different keys.

A map is useful if there is data and we wish to perform operations on the basis of the key. This map interface is implemented by various classes like [HashMap](#), [TreeMap](#), etc. Since all the subclasses implement the map, we can instantiate a map object with any of these classes.

For example:

```
Map<T> hm = new HashMap<> ();
```

```
Map<T> tm = new TreeMap<> ();
```

Where T is the type of the object.

Collection classes-

The Collection Classes The standard collection classes are

- 1) **The ArrayList Class** The ArrayList class extends AbstractList and implements the List interface. ArrayList is a generic class that has this declaration:
class ArrayList <E>

Here, E specifies the type of objects that the list will hold. ArrayList supports dynamic arrays that can grow as needed. an ArrayList is a variable-length array of object references. That is, an ArrayList can dynamically increase or decrease in size. lists are created with an initial size.

ArrayList has the constructors shown here:

ArrayList()

ArrayList(Collection c)

ArrayList(int capacity)

The first constructor builds an empty array list. The second constructor builds an array list that is initialized with the elements of the collection c. The third constructor builds an array list that has the specified initial capacity.

2) **The LinkedList Class** The LinkedList class extends AbstractSequentialList and implements the List, Deque, and Queue interfaces. It provides a linked-list data structure. LinkedList is a generic class that has this declaration:

class LinkedList<E>

Here, E specifies the type of objects that the list will hold. LinkedList has the two constructors shown here:

LinkedList()

LinkedList(Collection c)

The first constructor builds an empty linked list. The second constructor builds a linked list that is initialized with the elements of the collection c.

3) **The HashSet Class** HashSet extends AbstractSet and implements the Set interface. It creates a collection that uses a hash table for storage. HashSet is a generic class that has this declaration:

class HashSet <E>

Here, E specifies the type of objects that the set will hold.

The following constructors are defined: HashSet() HashSet(Collection c) HashSet(int capacity) HashSet(int capacity, float fillRatio) The first form constructs a default hash set. The second form initializes the hash set by using the elements of c. The third form initializes the capacity of the hash set to capacity. The fourth form initializes both the capacity and the fill ratio (also called load capacity) of the hash set from its arguments.

4) **The TreeSet Class** TreeSet extends AbstractSet and implements the NavigableSet interface. It creates a collection that uses a tree for storage. Objects are stored in sorted, ascending order. Access and retrieval times are quite fast, which makes TreeSet an excellent choice when storing large amounts of sorted information that must be found quickly. TreeSet is a generic class that has this declaration:

class TreeSet<E>

Here, E specifies the type of objects that the set will hold. TreeSet has the following constructors:

TreeSet()

TreeSet(Collection c)

TreeSet(Comparator comp)

TreeSet(SortedSet ss)

5) **Java HashMap**

In the ArrayList, Arrays store items as an ordered collection, and you have to access them with an index number (int type).

A HashMap however, store items **in "key/value" pairs**, and you can access them by an index of another type (e.g. a String).

One object is used as a key (index) to another object (value).

Java HashMap Constructors

HashMap provides 4 constructors and the access modifier of each is public which are listed as follows:

HashMap()

It is the default constructor which creates an instance of HashMap with an initial capacity of 16 and a load factor of 0.75.

HashMap(int initialCapacity)

It creates a HashMap instance with a specified initial capacity and load factor of 0.75.

HashMap(int initialCapacity, float loadFactor)

It creates a HashMap instance with a specified initial capacity and specified load factor.

HashMap(Map map)

It creates an instance of HashMap with the same mappings as the specified map.

6) **Java TreeMap class**

Java TreeMap class is a red-black tree based implementation.

It provides an efficient means of storing **key-value pairs** in sorted order.

The important points about Java TreeMap class are:

Java TreeMap contains **values based on the key**.

It implements the **NavigableMap interface and extends AbstractMap class**.

Java TreeMap contains **only unique elements**.

Java TreeMap cannot **have a null key but can have multiple null values**.

Java TreeMap is **non synchronized**.

Java TreeMap **maintains ascending order**.

Methods of Collection interface

No.	Method	Description
1	<code>public boolean add(E e)</code>	It is used to insert an element in this collection.
2	<code>public boolean addAll(Collection c)</code>	It is used to insert the specified collection elements in the invoking collection.
3	<code>public boolean remove(Object element)</code>	It is used to delete an element from the collection.
4	<code>public boolean removeAll(Collection c)</code>	It is used to delete all the elements of the specified collection from the invoking collection.
5	<code>public int size()</code>	It returns the total number of elements in the collection.
6	<code>public void clear()</code>	It removes the total number of elements from the collection.
7	<code>public boolean contains(Object element)</code>	It is used to search an element.
8	<code>public boolean containsAll(Collection c)</code>	It is used to search the specified collection in the collection.
9	<code>public Iterator iterator()</code>	It returns an iterator.
10	<code>public Object[] toArray()</code>	It converts collection into array.
11	<code>public boolean isEmpty()</code>	It checks if collection is empty.
12	<code>default Spliterator<E> spliterator()</code>	It generates a Spliterator over the specified elements in the collection.
13	<code>public boolean equals(Object element)</code>	It matches two collections.

1. ArrayList class

ArrayList is a part of **collection framework** and is present in java.util package.

The **ArrayList** class extends **AbstractList** and implements the **List** interface. **ArrayList** is a generic class that has this declaration:

class ArrayList<E>

Here, **E** specifies the type of objects that the list will hold.

ArrayList supports dynamic arrays that can grow or shrink as needed.

In Java, standard arrays are of a fixed length. After arrays are created, they cannot grow or shrink, which means that you must know in advance how many elements an array will hold.

But, sometimes, you may not know until run time precisely how large an array you need. To handle this situation, the Collections Framework defines **ArrayList**. An **ArrayList** is a variable-length array of object references. That is, an **ArrayList** can dynamically increase or decrease in size.

Though, it may be slower than standard arrays but can be helpful in programs where lots of manipulation in the array is needed.

Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged. When objects are removed, the array can be shrunk.

- Can contain duplicate elements.
- Maintains insertion order.
- Non synchronized.
- Allows random access because array works at the index basis.
- Manipulation is slow because a lot of shifting needs to occur if any element is removed from the array list.

Constructors of Java ArrayList

Constructor	Description
ArrayList()	It is used to build an empty array list.
ArrayList(Collection c)	It is used to build an array list that is initialized with the elements of the collection c.
ArrayList(int capacity)	It is used to build an array list that has the specified initial capacity.

Methods of Java ArrayList

Method	Description
void add(int index, Object element)	It is used to insert the specified element at the specified position index in a list.
boolean addAll(Collection c)	It is used to append all of the elements in the specified collection to the end of this list.
void clear()	It is used to remove all of the elements from this list.
int lastIndexOf(Object o)	It is used to return the index in this list of the last occurrence of the specified element, or -1 if the list does not contain this element.
Object[] toArray()	It is used to return an array containing all of the elements in this list in the correct order.
boolean add(Object o)	It is used to append the specified element to the end of a list.
boolean addAll(int index, Collection c)	It is used to insert all of the elements in the specified collection into this list, starting at the specified position.
int indexOf(Object o)	It is used to return the index in this list of the first occurrence of the specified element, or -1 if the List does not contain this element.
void trimToSize()	It is used to trim the capacity of this ArrayList instance to be the list's current size.
void remove(Object o)	It is used to remove a matched object
void remove(int index)	It is used to remove the object at specified index
int size()	Returns the number of elements in the collection

void removeAll(Collection c)	Removes the specified collection from the list
Object get(int index)	Returns the element specified in the index
void set(int index, Object o)	Replace the element in index with specified object
boolean contains(Object)	checking if the specified element exists in the given list or not.
boolean isEmpty()	Returns true if this list contains no elements.
Spliterator<E> spliterator()	It is used to create spliterator over the elements in a list.
List<E> subList(int start, int end)	It is used to fetch all the elements lies within the given range.
protected void removeRange(int start, int end)	It is used to remove all the elements lies within the given range.

It is used

// Demonstrate ArrayList.

```
import
java.util.*; class
ArrayListDemo
{
    public static void main(String args[])
    {
        //ArrayList al = new ArrayList(); //non-generic
        ArrayList<String> al = new ArrayList<String>(); //generic
        typeSystem.out.println("Initial size of al: " +al.size());
        // Add elements to the array list.
        al.add("CHENNAI");
        al.add("AHMEDABA
D");
        al.add("BANGALOR
E");
        al.add("HYDERABA
D");
        al.add("DELHI");
        al.add("PATNA");

        //displaying list
        System.out.p
rintln( al);
        System.out.println("After adding at a position ");
        al.add(3, "KOLKATA");
```

```

        System.out.println("Size of ArrayList after additions: " +al.size());
// Display the array list.

System.out.println( al);

System.out.println("The index of BANGALORE " +al.indexOf("BANGALORE"));
// Remove elements from the array list.

        al.remove("DELH
I");al.remove(2);
        al.add(5, "HYDERABAD");

//the array list using for-
each System.out.println("The
Array List is");
        for(String name : al)
            System.out.println(name);

System.out.println("Thelast indexofHyderabad"
+al.lastIndexOf("HYDERABAD"));ArrayList<String> bl = new
ArrayList<String>();
        bl.add("MUMBAI");
        bl.add("BHOPAL");

        al.addAll(4,bl);

if(al.contains("BHOPA
L"))
        System.out.println("Bhopal Exist");

if(al.containsAll(bl))
        System.out.println("Mumbai Bhopal Exist");

System.out.println("Array list after adding another list ");
System.out.println(al);

System.out.println("Array list after deleting new list ");
        al.removeAll(bl);
        System.out.println(al);

        al.clear();
System.out.println("Size of array list after clear: " +al.size());
System.out.println("The elements after clear: " +al);

```

```
    }  
}
```

// Demonstrate ArrayList with Userdefined Objects.

```
import  
java.util.*;class  
Book  
{  
    int id;  
    String  
    name,author;int  
    quantity;  
    public Book(int id, String name, String author, int quantity)  
    {  
        this.id = id;  
        this.name =  
        name; this.author  
        = author;  
        this.quantity = quantity;  
    }  
}  
public class ArrayListDemo1  
{  
    public static void main(String[] args)  
    {  
        //Creating list of Books  
        ArrayList<Book> list =new ArrayList<Book>();  
        //Creating Books  
        Book b1=new Book(101,"Let us C","Yashwant  
Kanetkar",8); Book b2=new Book(102,"Data  
Communications","Forouzan",4);Book b3=new  
Book(103,"Operating System","Galvin",6);  
        //Adding Books to list  
        list.add(b1);  
        list.add(b2);  
        list.add(b3);  
        //Displaying books  
        System.out.println("Size is " +list.size());  
        list.remove(1);  
  
        System.out.println("After remove size " + list.size());  
  
        for(Book b:list)  
        {  
            System.out.println(b.id+" "+b.name+" "+b.author+" "+b.quantity);  
        }  
    }  
}
```

// Demonstrate ArrayList with Iterator.

```
import java.util.*;
```

```
public class
```

```
ArrayList3
```

```
{
    public static void main(String[] args)
    {
        List<String> ls = new
        ArrayList<String>();
        ls.add("Varsha");
        ls.add("Harsha")
        ;
        ls.add("Srihrasha
        ");
        ls.add("Srivarsha
        ");
        ls.add("harshith"
        );

        //Accessing ArrayList using
Iterator Iterator i = ls.iterator();
        System.out.println("The ArrayList elements
        are:");while (i.hasNext())
        {
            System.out.println(i.next());
        }
    }
}
```

// Demonstrate ArrayList with Spliterator.

```
import java.util.*;
```

```
public class
```

```
ArrDemo4
```

```
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<String>();

        list.add("Hyderab
        d");
        list.add("Pune");
        list.add("Bangalo
        re");
    }
}
```

```

list.add("Kolkata
");
list.add("Chennai
");

Spliterator<String> city =

list.spliterator();

System.out.println("list of

Cities:");

// forEachRemaining method of Spliterator
city.forEachRemaining((n) -> System.out.println(n));
}
}

// Demonstrate ArrayList with Userdefined Objects.

public class EmpArrayList
{
    public static void main(String[] args)
    {
        //Creating list of Employees
        ArrayList<Employee> list =new ArrayList<Employee>();
        //Creating Employees
        Employee e1=new
        Employee(101,"A.Harsha",75000.50); Employee
        e2=new Employee(102,"B.Varsha",85000.50);
        Employee e3=new
        Employee(103,"C.Sirisha",95000.50); Employee
        e4=new Employee(104,"D.Sandeep",195000.50);

        //Adding Employees
        to list
        list.add(e1);
        list.add(e2);
        list.add(e3);
        list.add(e4);

        //Displaying Number of Employees
        System.out.println("\n The number of employees is" +list.size());

        //Displaying Details of Employees
        System.out.println("\n The employess data is
        \n");
        for(Employee e:list)
        {

```



```

        System.out.println(e.eid+":"+e.ename+":"+e.sal);
        System.out.println();
    }

    //Deleting an Employee

    list.remove(2);
    System.out.println("\n After removing number of employees are" +
        list.size());
    //Displaying Details of Employees

    System.out.println("\n The employess data after
        removing is \n");for(Employee e:list)
    {
        System.out.println(e.eid+":"+e.ename+":"+e.sal);
        System.out.println();
    }
}
}

```

2. LinkedList class

Java LinkedList class uses a doubly linked list to store the elements. It provides a linked-list data structure. It inherits the AbstractList class and implements List and Deque interfaces.

- o Can contain duplicate elements.
- o Maintains insertion order.
- o Non synchronized.
- o Manipulation is fast because no shifting needs to occur.
- o Can be used as a list, stack or queue.

Constructors of Java LinkedList

Constructo r	Descriptio n
LinkedList()	It is used to construct an empty list.
LinkedList(Collection c)	It is used to construct a list containing the elements of the specified collection,in the order, they are returned by the collection's iterator.

Methods of LinkedList

Method	Description
boolean add(E e)	It is used to append the specified element to the end of a list.
void add(int index, E element)	It is used to insert the specified element at the specified position index in a list.
boolean addAll(Collection c)	It is used to append all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
boolean addAll(int index, Collection c)	It is used to append all the elements in the specified collection, starting at the specified position of the list.
void addFirst(E e) void offerFirst()	It is used to insert the given element at the beginning of a list.
void addLast(E e) void offerLast()	It is used to append the given element to the end of a list.
void clear()	It is used to remove all the elements from a list.
boolean contains(Object o)	It is used to return true if a list contains a specified element.
E element()	It is used to retrieve the first element of a list.

E get(int index)	It is used to return the element at the specified position in a list.
E getFirst() peekFirst()	It is used to return the first element in a list.
E getLast() peekLast()	It is used to return the last element in a list.
int indexOf(Object o)	It is used to return the index in a list of the first occurrence of the specified element, or -1 if the list does not contain any element.
int lastIndexOf(Object o)	It is used to return the index in a list of the last occurrence of the specified element, or -1 if the list does not contain any element.
ListIterator<E> listIterator(int index)	It is used to return a list-iterator of the elements in proper sequence, starting at the specified position in the list.
E pop()	It pops an element from the stack represented by a list.
void push(E e)	It pushes an element onto the stack represented by a list.
E remove()	It is used to retrieve and removes the first element of a list.
E remove(int index)	It is used to remove the element at the specified position in a list.
boolean remove(Object o)	It is used to remove the first occurrence of the specified element in a list.
E removeFirst() pollFirst()	It removes and returns the first element from a list.

Boolean removeFirstOccurrence(Object)	It is used to remove the first occurrence of the specified element in a list (when traversing the list from head to tail).
E removeLast() pollLast()	It removes and returns the last element from a list.
boolean removeLastOccurrence(Object)	It removes the last occurrence of the specified element in a list (when traversing the list from head to tail).
E set(int index, E element)	It replaces the element at the specified position in a list with the specified element.
Object[] toArray()	It is used to return an array containing all the elements in a list in proper sequence (from first to the last element).
int size()	It is used to return the number of elements in a list.

```
//Program to Demonstrate LinkedList
import java.util.*;
public class LinkedList1
{
    public static void main(String args[])
    {
        LinkedList<String> ls = new
        LinkedList<String>();ls.add("B");
        ls.add("C");
        ls.add("D");
        ls.add("E");

        System.out.println("The List is " +ls);

        ls.add(2,"F"); //add at index 2
        System.out.println("The List after add
        is " +ls);

        ls.set(2,"R");
        System.out.println("The List after set is " +ls);
    }
}
```

```
LinkedList<String> ls1 = new  
LinkedList<String>();ls1.add("P");  
ls1.add("Q");
```

```
ls.addAll(ls1);  
System.out.println("List after adding other list is " +ls);
```

```
LinkedList<String> ls2 = new  
LinkedList<String>();ls2.add("X");  
ls2.add("Y");
```

```
ls.addAll(4,ls2);  
System.out.println("List after adding other list at given index is " +ls);
```

```
ls.add("D");  
ls.add("R");  
ls.addFirst("A");  
ls.addLast("Z");
```

```
System.out.println("The List after addind at first and last is "  
+ls);if(ls.contains("Z"))  
System.out.println("Z exist");
```

```
System.out.println("The first element of the list is " +ls.element());  
System.out.println("The first element of the list is " +ls.getFirst());  
System.out.println("The element at 6 place is "+ls.get(5));  
System.out.println("The Last element of list is " +ls.getLast());  
System.out.println("The index of F element is "+ls.indexOf("F"));
```

```
ls.push("M");
```

```
System.out.println("The List after push is "  
+ls); System.out.println("The Popped Element  
is " +ls.pop());ls.remove();
```

```
ls.remove(3)  
;  
ls.remove("  
P");  
ls.removeFirstOccurrence
```

```
("D");  
ls.removeLastOccurrence  
("R");
```

```
System.out.println("The final list is "+ls);  
System.out.println("The size of the list finally : "  
+ls.size());
```

```
Iterator i = ls.iterator();  
System.out.println("The Linked List  
elements are:"); while (i.hasNext())  
    {  
        System.out.println(i.next());  
    }
```

```
/* Spliterator<String> list = ls.spliterator();
```

```
System.out.println("list of letters using spliterator:");
```

```
// forEachRemaining method of Spliterator  
list.forEachRemaining((n) -> System.out.println(n)); */
```

```
ls.clear();  
System.out.println("The list after clear "+ls);
```

```
    }  
}
```

3. HashSet class

HashSet class is used to create a collection that uses a hash table for storage. It inherits the AbstractSet class and implements Set interface.

- o Stores the elements by using a mechanism called hashing.
- o Contains unique elements only.
- o Allows null value.
- o Non synchronized.
- o HashSet doesn't maintain the insertion order. Here, elements are inserted on the basis of their hashCode.
- o HashSet is the best approach for search operations.

Constructors of Java HashSet class

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(int capacity)	It is used to initialize the capacity of the hash set to the given integer value capacity. The capacity grows automatically as elements are added to the HashSet.

Methods of Java HashSet class

Modifier & Type	Method	Description
boolean	<u>add(E e)</u>	It is used to add the specified element to this set if it is not already present.
void	<u>clear()</u>	It is used to remove all of the elements from the set.
boolean	<u>contains(Object o)</u>	It is used to return true if this set contains the specified element.
boolean	<u>isEmpty()</u>	It is used to return true if this set contains no elements.

Iterator<E>	<u>iterator()</u>	It is used to return an iterator over the elements in this set.
boolean	<u>remove(Object o)</u>	It is used to remove the specified element from this set if it is present.
int	<u>size()</u>	It is used to return the number of elements in the set.
Spliterator<E>	<u>spliterator()</u>	It is used to create a late-binding and fail-fast Spliterator over the elements in the set.
public boolean	<u>addAll(Collection c)</u>	It is used to insert the specified collection elements in the invoking collection.
public boolean	<u>removeAll(Collection c)</u>	It is used to delete all the elements of the specified collection from the invoking collection.
public void	<u>clear()</u>	It removes the total number of elements from the collection.
public boolean	<u>containsAll(Collection c)</u>	It is used to search the specified collection in the collection.

/* Use HashSet methods to perform operations on collection of data */

```

import
java.util.*;
class
HashDemo
{
    public static void main(String args[])
    {
        //Creating HashSet
        HashSet<String> set=new HashSet<String>();

        //Adding Elements to HashSet - ignores
        //duplicates
        set.add("hyderabad");
        set.add("hyderaba
d");
    }
}

```



```

set.add("bangalore");
set.add("chennai");

set.add("kolkata");
set.add("kolkata");
set.add("pune");

System.out.println("The number of cities is " +
set.size());if(set.contains("chennai"))
    System.out.println("Tamil Language");

```

```

Iterator<String>
i=set.iterator();
while(i.hasNext())
{
    System.out.println(i.next());
}

```

```

//Removing specific element from
HashSetset.remove("hyderabad");

```

```

//Displaying set
System.out.println("\n The list after remove -> "+set);

```

```

//adding another set

```

```

HashSet<String> set1=new
    HashSet<String>();
    set1.add("Panjab");
    set1.add("Delhi");
    set.addAll(set1);

```

```

System.out.println("\n Updated after adding List is -> "+set);

```

```

//removing new set
from list

```

```

set.removeAll(set1);

```

```

System.out.println("\n Updated List after removing list is -> "+set);

```

```

//Removing all the elements available in the

```

```

setset.clear();

```

```

System.out.println("\n After clear the set is -> "+set);

```

```

}

```

```

}

```

4. TreeSet class

TreeSet class implements the **Set** interface that uses a tree for storage. It inherits AbstractSet class and implements the NavigableSet interface. The objects of the TreeSet class are stored in sorted ascending order.

- o Contains unique elements only like HashSet.
- o Access and retrieval times are quite fast.
- o Doesn't allow null element.
- o Non synchronized.
- o Maintains ascending order.

Constructors of Java TreeSet class

Constructor	Description
TreeSet()	It is used to construct an empty tree set that will be sorted in ascending order according to the natural order of the tree set.
TreeSet(Collection c)	It is used to build a new tree set that contains the elements of the collection c.

Methods of Java TreeSet class

Method	Description
boolean add(E e)	It is used to add the specified element to this set if it is not already present.
boolean addAll(Collection c)	It is used to add all of the elements in the specified collection to this set.
Iterator descendingIterator()	It is used to iterate the elements in descending order.
E floor(E e)	It returns the equal or closest least element of the specified element from the set, or null if there is no such element.
SortedSet headSet(E toElement)	It returns the group of elements that are less than the specified element.

E higher(E e)	It returns the closest greatest element of the specified element from the set, or null there is no such element.
Iterator iterator()	It is used to iterate the elements in ascending order.
E lower(E e)	It returns the closest least element of the specified element from the set, or null there is no such element.
E pollFirst()	It is used to retrieve and remove the lowest(first)element.
E pollLast()	It is used to retrieve and remove the highest(last)element.
SortedSet subSet(E fromElement, E toElement))	It returns a set of elements that lie between the given range which includes fromElement and excludes toElement.
SortedSet tailSet(E fromElement)	It returns a set of elements that are greater than or equal to the specified element.
boolean contains(Object o)	It returns true if this set contains the specified element.
boolean isEmpty()	It returns true if this set contains no elements.
boolean remove(Object o)	It is used to remove the specified element from this set if it is present.
void clear()	It is used to remove all of the elements from this set.
E first()	It returns the first (lowest) element currently in this sorted set.
E last()	It returns the last (highest) element currently in this sorted set.
int size()	It returns the number of elements in this set.

//Program to Demonstrate TreeSet

```
import java.util.*;
public class
TreeDemo
{
    public static void main(String args[])

    {
        TreeSet<String> ts = new
        TreeSet<String>();ts.add("M");
                                ts.add("G");
        ts.add("C");      ts.add("Z");
        ts.add("W") ;    ts.add("I");
        System.out.println("The tree set
        is " +ts);

        TreeSet<String> ts1 = new
        TreeSet<String>();ts1.add("A");
                                ts1.add("K");
        ts.addAll(ts1);
        System.out.println("The adding collection is " +ts);

        Iterator i = ts.iterator();
        System.out.println("The Tree set elements
        are:");while (i.hasNext())
        {
            System.out.println(i.next());
        }

        Iterator p = ts.descendingIterator();
        System.out.println("The Descending Tree set
        elements are:");while (p.hasNext())
        {
            System.out.println(p.next());
        }

        System.out.println("The item lower than C is " +ts.lower("C"));
        System.out.println("The item higer than W is " +ts.higher("W"));

        System.out.println("The sub set is " + ts.subSet("G" ,"M"));
        //G inclusive and M exclusive

        System.out.println("The list from I is " +ts.tailSet("I"));
        System.out.println("The list up to I is "
        +ts.headSet("I"));
```

```

        System.out.println("The first element of list is "
            +ts.first()); System.out.println("The last element
            of list is " +ts.first()); System.out.println("The
            first one removed " +ts.pollFirst());
        System.out.println("The last one removed "
            +ts.pollLast());

        System.out.println("The size of list is "
            +ts.size()); System.out.println("The final
            list is "+ts); ts.clear();
        System.out.println("The list after clear " +ts);

    }
}

```

5. Date

The `java.util.Date` class represents date and time in java. It provides constructors and methods to deal with date and time in java.

java.util.Date Constructors

Constructor	Description
<code>Date()</code>	Creates a date object representing current date and time.
<code>Date(long milliseconds)</code>	Creates a date object for the given milliseconds since January 1, 1970, 00:00:00 GMT.

java.util.Date Methods

Method	Description
<code>boolean after(Date date)</code>	tests if current date is after the given date.
<code>boolean before(Date date)</code>	tests if current date is before the given date.
<code>int compareTo(Date date)</code>	compares current date with given date.
<code>boolean equals(Date date)</code>	compares current date with given date for equality.
<code>long getTime()</code>	returns the time represented by this date object.

int hashCode()	returns the hash code value for this date object.
void setTime(long time)	changes the current date and time to given time.
String toString()	converts this date into Instant object.

//Example Date class Methods

```
import
java.util.Date;
public class
DateDemo
{
    public static void main(String args[])
    {
        // Instantiate a Date
        objectDate d1= new
        Date();

        System.out.println("The current Date and Time is " +d1.toString());

        long msec =
        System.currentTimeMillis(); Date
        d2 = new Date(msec);
        System.out.println("Using second Constructor " + d2);

        if(d1.equals(d2))
            System.out.println("The Two Date objects are equal");
        else
            System.out.println("The Two Date objects are not equal");

        System.out.println("Get the time of Date d1 object "
        +d1.getTime());System.out.println("Get the time of Date
        d2 object " +d2.getTime());

        int x =
        d2.compareTo(d1);
        Date d3 = new Date();
        d3.setTime(12565589
        92);

        System.out.println("The Date object d3 Date and Time is " +
        d3.toString());System.out.println("The dates
        comparision is " +x);

        if(d2.after(d3))
```

```
        System.out.println(" date3 is old");
    else
        System.out.println(" date2 is old");

    }
}
```

6. Java HashMap

- In the ArrayList, Arrays store items as an ordered collection, and you have to access them with an index number (int type).
- A HashMap however, store items in "key/value" pairs, and you can access them by an index of another type (e.g. a String).
- One object is used as a key (index) to another object (value).
- It can store different types: String keys and Integer values, or the same type, like:
- String keys and String values:
- Java HashMap contains values based on the key.
- Java HashMap contains only unique keys.
- Java HashMap may have one null key and multiple null values.
- Java HashMap is non synchronized.
- Java HashMap maintains no order.
- The initial default capacity of Java HashMap class is 16

Java HashMap Constructors

HashMap provides 4 constructors and the access modifier of each is public which are listed as follows:

HashMap()

It is the default constructor which creates an instance of HashMap with an initial capacity of 16 and a load factor of 0.75.

HashMap(int initialCapacity)

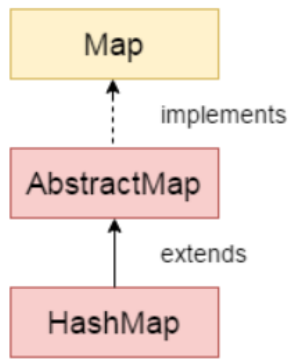
It creates a HashMap instance with a specified initial capacity and load factor of 0.75.

HashMap(int initialCapacity, float loadFactor)

It creates a HashMap instance with a specified initial capacity and specified load factor.

HashMap(Map map)

It creates an instance of HashMap with the same mappings as the specified map



Characteristics of Java HashMap:

- A HashMap is a data structure that is used to store and retrieve values based on keys. Some of the key characteristics of a hashmap include:
- Fast access time: HashMaps provide constant time access to elements, which means that retrieval and insertion of elements are very fast, usually $O(1)$ time complexity.
- Uses hashing function: HashMaps use a hash function to map keys to indices in an array. This allows for a quick lookup of values based on keys.
- Stores key-value pairs: Each element in a HashMap consists of a key-value pair. The key is used to look up the associated value.
- Supports null keys and values: HashMaps allow for null values and keys. This means that a null key can be used to store a value, and a null value can be associated with a key.
- Not ordered: HashMaps are not ordered, which means that the order in which elements are added to the map is not preserved.
- Allows duplicates: HashMaps allow for duplicate values, but not duplicate keys. If a duplicate key is added, the previous value associated with the key is overwritten.
- Thread-unsafe: HashMaps are not thread-safe, which means that if multiple threads access the same hashmap simultaneously, it can lead to data inconsistencies.
- Capacity and load factor: HashMaps have a capacity, which is the number of elements that it can hold, and a load factor, which is the measure of how full the hashmap can be before it is resized.

Method	Description
<u>clear()</u>	Remove all entries from the map.
<u>clone()</u>	Create a copy of the HashMap.
<u>compute()</u>	Compute a value for an entry based on its key and the current value (if it has one)
<u>computeIfAbsent()</u>	Compute a value for an entry based on its key only if an entry using the key does not already exist
<u>computeIfPresent()</u>	Compute a new value for an entry based on its key and current value but only if an entry with the key already exists
<u>containsKey()</u>	Indicate if an entry with the specified key exists in the map
<u>containsValue()</u>	Indicate if an entry with the specified value exists in the map
<u>entrySet()</u>	Return a set of all entries in the map

<u>forEach()</u>	Perform an action on every entry in the map
<u>get()</u>	Return the value of the entry with a specified key
<u>getOrDefault()</u>	Return the value of the entry with a specified key or a default value if the entry is not found
<u>isEmpty()</u>	Indicate whether the map is empty
<u>keySet()</u>	Return a set of all keys in the map
<u>merge()</u>	Compute a value for an entry based on its key and value or write a specific value if the entry does not yet exist
<u>put()</u>	Write an entry into the map
<u>putAll()</u>	Write all of the entries from another map into this one
<u>putIfAbsent()</u>	Write an entry into the map but only if an entry with the same key does not already exist
<u>remove()</u>	Remove an entry from the map
<u>replace()</u>	Write to an entry in the map only if it exists
<u>replaceAll()</u>	Replaces the value of every entry with the result of an operation
<u>size()</u>	Return the number of entries in the map
<u>values()</u>	Return a collection containing all of the values in the map

Example. // Import the HashMap class

```
import java.util.HashMap;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Create a HashMap object called capitalCities
```

```
HashMap<String, String> capitalCities = new HashMap<String, String>();
```

```
        // Add keys and values (Country, City)
```

```
        capitalCities.put("England", "London");
```

```
        capitalCities.put("Germany", "Berlin");
```

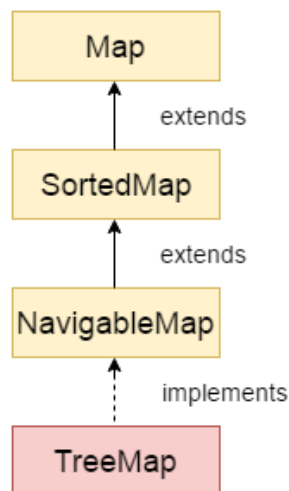
```
        capitalCities.put("Norway", "Oslo");
```

```
capitalCities.put("USA", "Washington DC");
System.out.println(capitalCities);
}
}
```

OUTPUT : {USA=Washington DC, Norway=Oslo, England=London, Germany=Berlin}

7) Java TreeMap class

- Java TreeMap class is a red-black tree based implementation.
 - It provides an efficient means of storing key-value pairs in sorted order.
- The important points about Java TreeMap class are:
- Java TreeMap contains values based on the key.
 - It implements the NavigableMap interface and extends AbstractMap class.
 - Java TreeMap contains only unique elements.
 - Java TreeMap cannot have a null key but can have multiple null values.
 - Java TreeMap is non synchronized.
 - Java TreeMap maintains ascending order.



Sr.No.	Constructors & Description
--------	----------------------------

	TreeMap()
--	-------------------

- | | |
|---|--|
| 1 | This constructor constructs an empty tree map that will be sorted using the natural order of its keys. |
|---|--|

	<u>TreeMap(Comparator comp)</u>
--	--

- | | |
|---|--|
| 2 | This constructor constructs an empty tree-based map that will be sorted using the Comparator comp. |
|---|--|

	<u>TreeMap(Map m)</u>
--	------------------------------

- | | |
|---|--|
| 3 | This constructor initializes a tree map with the entries from m , which will be sorted using the natural order of the keys. |
|---|--|

	<u>TreeMap(SortedMap sm)</u>
--	-------------------------------------

- | | |
|---|---|
| 4 | This constructor initializes a tree map with the entries from the <u>SortedMap sm</u> , which will be sorted in the same order as sm . |
|---|---|

Sr.No.	Method & Description
--------	----------------------

	void clear()
--	---------------------

- | | |
|---|---|
| 1 | Removes all mappings from this TreeMap. |
|---|---|

	Object clone()
--	-----------------------

- | | |
|---|--|
| 2 | Returns a shallow copy of this TreeMap instance. |
|---|--|

	Comparator comparator()
--	--------------------------------

- | | |
|---|--|
| 3 | Returns the comparator used to order this map, or null if this map uses its keys' natural order. |
|---|--|

	boolean containsKey(Object key)
--	--

- | | |
|---|--|
| 4 | Returns true if this map contains a mapping for the specified key. |
|---|--|

	<u>boolean containsValue(Object value)</u>
--	---

- | | |
|---|--|
| 5 | Returns true if this map maps one or more keys to the specified value. |
|---|--|

- Set entrySet()**
 6 Returns a set view of the mappings contained in this map.
- Object firstKey()**
 7 Returns the first (lowest) key currently in this sorted map.
- Object get(Object key)**
 8 Returns the value to which this map maps the specified key.
- SortedMap headMap(Object toKey)**
 9 Returns a view of the portion of this map whose keys are strictly less than toKey.
- Set keySet()**
 10 Returns a Set view of the keys contained in this map.
- Object lastKey()**
 11 Returns the last (highest) key currently in this sorted map.

- Object put(Object key, Object value)**
 12 Associates the specified value with the specified key in this map.
- void putAll(Map map)**
 13 Copies all of the mappings from the specified map to this map.
- Object remove(Object key)**
 14 Removes the mapping for this key from this TreeMap if present.
- int size()**
 15 Returns the number of key-value mappings in this map.
- SortedMap subMap(Object fromKey, Object toKey)**
 16 Returns a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive.

TREEMAP EXAMPLE

```
import java.util.TreeMap;
class Main {
    public static void main(String[] args) {
        // Creating TreeMap of even numbers
        TreeMap<String, Integer> evenNumbers = new TreeMap<>();
        // Using put()
        evenNumbers.put("Two", 2);
        evenNumbers.put("Four", 4);
        // Using putIfAbsent()
        evenNumbers.putIfAbsent("Six", 6);
        System.out.println("TreeMap of even numbers: " + evenNumbers);
        //Creating TreeMap of numbers
        TreeMap<String, Integer> numbers = new TreeMap<>();
        numbers.put("One", 1);
        // Using putAll()
        numbers.putAll(evenNumbers);
        System.out.println("TreeMap of numbers: " + numbers);
    }
}
```

output

TreeMap of even numbers: {Four=4, Six=6, Two=2}

TreeMap of numbers: {Four=4, One=1, Six=6, Two=2}